

Architecture de l'environnement de développement Docker avec orchestration Kubernetes

Pialoa Tech
Équipe de développement

24 juillet 2026

Résumé

Ce document décrit l'architecture de l'environnement de développement Docker pour l'application Pialoa Tech, ainsi que la façon dont Kubernetes peut être utilisé comme orchestrateur pour gérer les conteneurs en production ou en environnements de staging. Nous détaillons les services, les volumes, les réseaux et les bonnes pratiques pour assurer une isolation, une reproductibilité et une évolutivité optimales.

1 Introduction

L'application Pialoa Tech est une application web développée avec Laravel 13, comportant un site vitrine public et un espace d'administration protégé par authentification (e-mail + mot de passe) avec des fonctionnalités CRUD sur les Produits, Services, Événements et Stagiaires. Pour garantir un environnement de développement cohérent et isolé, nous avons adopté Docker Compose pour l'orchestration locale des services. Dans une perspective de mise à l'échelle et de haute résilience, les mêmes images peuvent être déployées sur un cluster Kubernetes.

2 Architecture Docker Compose

2.1 Services principaux

Service	Description
app	Conteneur PHP-FPM 8.2 avec les extensions nécessaires (pdo_mysql, mbstring, gd, etc.), Node.js,
db	Base de données MySQL 8.0 initialisée avec la base <code>pialoatech</code> et l'utilisateur <code>laravel</code> .

TABLE 1 – Services définis dans `docker-compose.yml`

2.1.1 Service app

- **Image** : construite à partir du `Dockerfile` présent à la racine du projet.
- **Ports exposés** :
 - 8000:8000 – serveur de développement Laravel (`php artisan serve`).
 - 5173:5173 – serveur de développement Vite (`npm run dev`).
- **Volumes** : montage en cache du répertoire projet (`./:/var/www:cached`) afin de refléter instantanément les modifications du code.
- **Variables d'environnement** : connexion à la base de données via le nom d'hôte `db` (service Docker), compatible avec le fichier `.env`.

- **Dépendances** : dépend du service `db` pour s’assurer que MySQL est prêt avant le démarrage de l’application.

2.1.2 Service db

- **Image** : `mysql:8.0` officielle.
- **Port** : `3306:3306` pour permettre l’accès externe (outils clients).
- **Volumes** : volume nommé `db_data` persistant les données MySQL entre les redémarrages.
- **Variables d’environnement** : définition de la base de données, de l’utilisateur et du mot de passe.
- **Commande** : utilisation du plugin d’authentification `mysql_native_password` pour garantir la compatibilité avec les clients PHP.

2.2 Réseaux

Par défaut, Docker Compose crée un réseau privé nommé `pialotech_app_default` auquel tous les services sont attachés, permettant la communication inter-service via les noms de service (`app`, `db`) comme noms d’hôte.

2.3 Volumes persistants

- `db_data` : stocke les fichiers de données MySQL afin de préserver l’état entre les arrêts/-conteneurs.
- (Optionnel) `node_modules` : pourrait être monté en volume pour accélérer les réinstallations de dépendances npm, mais dans notre configuration nous préférons reconstruire à chaque build pour garantir la cohérence.

2.4 Workflow de développement

1. `cp .env.example .env` – copie de la configuration d’exemple.
2. `docker-compose up -d` – lancement des conteneurs en arrière-plan.
3. `docker-compose exec app composer install` – installation des dépendances PHP (si non encore présentes).
4. `docker-compose exec app npm install` – installation des dépendances JavaScript.
5. `docker-compose exec app php artisan migrate -seed` – exécution des migrations et des seeders.
6. `docker-compose exec app php artisan storage:link` – création du lien de stockage public.
7. L’application est accessible sur `http://localhost:8000` et les assets sont hot-reloaded via Vite sur le port 5173.

3 Passage à Kubernetes (orchestration)

3.1 Raison d’être

En production, ou pour des environnements de staging nécessitant une haute disponibilité, du scaling horizontal et des stratégies de déploiement avancées (rolling updates, blue/green, canary), Kubernetes offre un orchestrateur de conteneurs robuste.

3.2 Manifestes Kubernetes de base

Nous proposons deux ressources principales : un **Deployment** pour l'application et un **StatefulSet** (ou **Deployment** avec un volume persistant) pour la base de données MySQL.

3.2.1 Déploiement de l'application (app-deployment.yaml)

Listing 1 – Déploiement Kubernetes de l'application

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pialoa-app
  labels:
    app: pialoa
spec:
  replicas: 2
  selector:
    matchLabels:
      app: pialoa
  template:
    metadata:
      labels:
        app: pialoa
    spec:
      containers:
        - name: app
          image: pialoa/app:latest
          ports:
            - containerPort: 8000
            - containerPort: 5173
          envFrom:
            - configMapRef:
                name: pialoa-app-config
          volumeMounts:
            - name: app-code
              mountPath: /var/www
          # Liveness/Readiness probes
          livenessProbe:
            httpGet:
              path: /up
              port: 8000
            initialDelaySeconds: 30
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /up
              port: 8000
            initialDelaySeconds: 5
            periodSeconds: 5
      volumes:
        - name: app-code
```

```
emptyDir: {}
```

Dans cet exemple, nous utilisons 2 réplicas pour la haute disponibilité, et le conteneur expose les ports 8000 (Laravel) et 5173 (Vite).

3.2.2 StatefulSet pour MySQL (mysql-statefulset.yaml)

Listing 2 – StatefulSet Kubernetes pour MySQL

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: pialoa-mysql
spec:
  serviceName: "pialoa-mysql"
  replicas: 1
  selector:
    matchLabels:
      app: pialoa-mysql
  template:
    metadata:
      labels:
        app: pialoa-mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          ports:
            - containerPort: 3306
          env:
            - name: MYSQL_DATABASE
              value: "pialoatech"
            - name: MYSQL_USER
              value: "laravel"
            - name: MYSQL_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: pialoa-mysql-secret
                  key: password
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: pialoa-mysql-secret
                  key: root-password
          volumeMounts:
            - name: mysql-data
              mountPath: /var/lib/mysql
      volumeClaimTemplates:
        - metadata:
            name: mysql-data
          spec:
            accessModes: [ "ReadWriteOnce" ]
            resources:
```

```
requests:
  storage: 10Gi
```

Ici, nous définissons un StatefulSet avec un seul réplica et un volume persistant de 10 Go pour les données MySQL.

3.2.3 ConfigMaps et Secrets

- **ConfigMap** `pialoa-app-config` : contient les variables d'environnement non sensibles (`APP_NAME`, `APP_ENV`, etc.).
- **Secret** `pialoa-mysql-secret` : stocke `MYSQL_PASSWORD` et `MYSQL_ROOT_PASSWORD` encodés en base64.

3.2.4 Service exposant l'application

Listing 3 – Service Kubernetes pour exposer l'application

```
apiVersion: v1
kind: Service
metadata:
  name: pialoa-app
spec:
  type: LoadBalancer # ou NodePort/Ingress selon l'environnement
  selector:
    app: pialoa
  ports:
    - name: http
      port: 80
      targetPort: 8000
    - name: vite
      port: 80
      targetPort: 5173
```

Ce service expose l'application sur le port 80 (redirigé vers le port 8000 pour Laravel) et le port 80 pour Vite (redirigé vers le port 5173).

3.3 Stratégie de déploiement

1. Construire les images Docker et les pousser vers un registre (ex. GitHub Packages, Docker Hub, ou un registre privé).
2. Appliquer les manifests Kubernetes :
 - `kubectl apply -f mysql-statefulset.yaml`
 - `kubectl apply -f app-deployment.yaml`
 - `kubectl apply -f app-service.yaml`
3. Vérifier l'état des pods avec `kubectl get pods`.
4. Accéder à l'application via l'adresse externe fournie par le LoadBalancer ou via un Ingress configuré avec un nom de domaine (ex. `dev.pialoa.tech`).

3.4 Considérations spécifiques

- **Persistence** : le volume `mysql-data` garantit que les données de la base de données survivent aux redémarrages de pods.

- **Network Policies** : limiter l'accès au port 3306 uniquement au pod de l'application.
- **Monitoring & Logging** : intégrer Prometheus/Grafana pour les métriques et ELK ou Loki pour les logs.
- **CI/CD** : automatiser la construction d'images, le push vers le registre et le déploiement via ArgoCD ou GitHub Actions.

4 Conclusion

L'environnement de développement Docker décrit ci-dessus offre une reproduction rapide, isolée et cohérente de l'application Pialoa Tech grâce à Docker Compose. Pour évoluer vers une architecture de production résiliente et scalable, Kubernetes constitue l'orchestrateur de choix, permettant le déploiement de conteneurs avec gestion du trafic, scaling automatique et persistance des données. La transition de Docker Compose à Kubernetes peut être réalisée progressivement en réutilisant les mêmes images Docker et en adaptant la configuration via des manifests Kubernetes (Deployments, StatefulSets, Services, ConfigMaps, Secrets).